Davante

20

PROGRAMACIÓN

Bases de datos orientadas a objetos

ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. Persistencia de datos 5

/ 3. Técnicas de persistencia 6

3.1. Persistencia nativa 6

3.2. DAO 6

3.3. Framework de persistencia: JPA 6

/ 4. Caso práctico 1: "Entendiendo las bases de datos orientadas a objetos" 7

/ 5. Entidades 7

/ 6. Unidad de persistencia 8

/ 7. Gestor de entidades 9

7.1. EntityManagerFactory 9

7.2. EntityManager 10

7.3. Métodos y transacciones del EntityManager 10

/ 8. Operaciones CRUD 11

8.1. Create 12

8.2. Read 12

8.3. Update 12

8.4. Delete 13

ÍNDICE

/ 9. Controlador JPA 13

9.1. Componentes de un controlador JPA 13

/ 10. Consulta de entidades: JPQL 14

/ 11. Caso práctico 2: "¿Se puede seguir empleando SQL?" 15

/ 12. Consultas JPQL 16

12.1. Select 16

12.2. Consultas Update y Delete 17

12.3. Otras consultas 17

/ 13. Resumen y resolución del caso práctico de la unidad 18

/ 14. Bibliografía 19

© MEDAC ISBN: 978-84-19885-69-2
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Definir la importancia de los mecanismos de conversión entre datos mediante objetos.

Instalar un gestor de bases de datos orientadas a objetos.

Realizar programas con acceso a bases de datos orientadas a objetos.

Insertar datos en una base de datos orientada a objetos.

Recuperar información de una base de datos orientada a objetos.

Borrar y modificar datos de una base de datos orientada a objetos.

/ 1. Introducción y contextualización práctica

En esta unidad vamos a estudiar un nuevo tipo de base de datos, las bases de datos orientadas a objetos, y sus características.

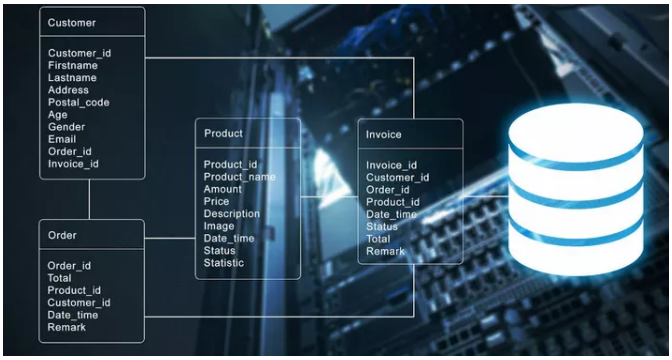


Fig.1. Bases de datos orientadas a objetos.

Vamos a ver qué gestores de bases de datos soportan este tipo de base de datos y cómo podemos utilizarlas dentro de nuestros proyectos.

Además, vamos a aprender a realizar las operaciones típicas de toda base de datos, como insertar, actualizar, borrar y consultar datos con diferentes criterios de búsqueda, teniendo en cuenta que no podemos emplear SQL, ya que se debe trabajar con entidades en el nuevo contexto con el que trabajan las bases de datos orientadas a objetos.

A continuación, se presenta un caso práctico a través del cual podremos aproximarnos de forma aplicada a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Audio intro. "Bases de datos orientadas a objetos vs. bases de datos relacional"

http://bit.ly/35vPx81

/ 2. Persistencia de datos

Hasta ahora, conocemos **dos formas** de persistencia de datos:

- **Ficheros**.
- **Servicios de un SGBD**.

Generalmente, se considera que los **SGBD** son **preferibles por** varias razones, entre las que se incluyen:

- **Integridad de los datos**: Los SGBD están diseñados para mantener la integridad de los datos, lo que significa que ofrecen mecanismos para garantizar que los datos almacenados cumplan con ciertas reglas o restricciones.
- **Transacciones**: Los SGBD admiten transacciones, lo que permite agrupar operaciones de manipulación de datos en unidades lógicas y asegurar que estas unidades se completen correctamente o se reviertan por completo si ocurre algún error. Esto garantiza la consistencia de los datos incluso en entornos concurrentes.
- **Escalabilidad y rendimiento**: Los SGBD están optimizados para manejar grandes volúmenes de datos y consultas complejas de manera eficiente. Utilizan índices, optimizadores de consultas y otras técnicas para mejorar el rendimiento de las consultas.
- **Concurrencia**: Los SGBD gestionan la concurrencia de manera eficiente, permitiendo que múltiples usuarios accedan a los mismos datos al mismo tiempo de forma segura y sin corrupción de datos.
- **Seguridad**: Los SGBD ofrecen mecanismos de seguridad integrados para proteger los datos, como la autenticación de usuarios, la autorización de acceso a los datos y el cifrado de datos.

La persistencia se encarga de guardar y recuperar datos de una aplicación, sin embargo, los datos manejados por una aplicación son objetos, y, por tanto, la manera de guardarlos y recuperarlos dependerá del SGBD empleado. Se pueden hacer dos distinciones:

- **SGBD orientado a objetos**: Estos tipos están diseñados específicamente para almacenar y manipular objetos de manera directa.
- **SGBD relacionales**: Trabajan principalmente con conjuntos de datos tabulares.

En ambos sistemas, se lleva a cabo la persistencia de datos, sin embargo, el uso de un SGBDR emplea registros de tablas que obligan a realizar una conversión entre objetos y registros de una tabla y viceversa.

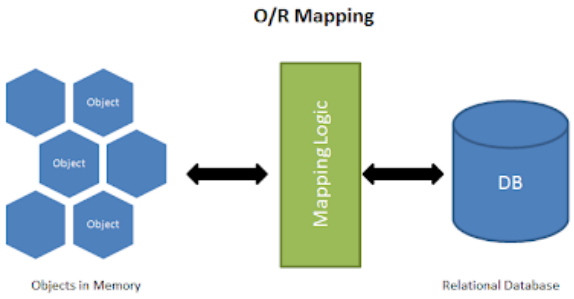


Fig.2. Mapeo objeto-relacional.

En el siguiente audio podrás aprender más sobre el apartado anterior.

Audio 1. "SGBDOO"
http://bit.ly/2LIY940

/ 3. Técnicas de persistencia

A la hora de realizar la persistencia en nuestro programa, hemos visto que existen distintos **métodos** que varían **en función del nivel de abstracción**:

3.1. Persistencia nativa

Se trata de **añadir**, a las clases, los **métodos que se encargan de guardar y recuperar la información de la base de datos**. Estos métodos, generalmente llamados save(), read() y update() emplean las consultas SQL.

Sin embargo, esta técnica rompe con la abstracción que la clase que lo implementa modela de la realidad, al mezclar la lógica de negocio y la lógica de acceso a datos.

3.2. DAO

Este patrón **se encarga de separar la lógica de negocio de la lógica de acceso a los datos en una aplicación**, con el fin de abstraer y encapsular todos los detalles relacionados con la recuperación y el almacenamiento de datos, permitiendo así un mejor manejo y mantenimiento del código.

El patrón DAO proporciona una interfaz entre la lógica de la aplicación y la capa de acceso a datos (en este caso una base de datos), por lo que las operaciones de acceso a datos, como insertar, actualizar, eliminar y recuperar datos, se encapsulan en clases DAO dedicadas.

3.3. Framework de persistencia: JPA

Otra alternativa es el uso de *frameworks* de persistencia **encargados de manejar los objetos**. En su día, comenzaron a aparecer distintos frameworks que, aunque tuviesen distintas implementaciones, tenían las mismas funcionalidades, por lo que se desarrolló una única especificación que uniera las implementaciones de los distintos fabricantes dando luz a JPA.



Fig.3. JPA.

JPA es una **API estándar de Java** que proporciona un marco de trabajo para mapear objetos Java a entidades de bases de datos relacionales.

Define un conjunto de anotaciones e interfaces que permiten a los desarrolladores trabajar con objetos Java de manera más natural, mientras que la persistencia de esos objetos en una base de datos relacional se maneja de forma transparente.

¡! Sabías que...

Existen varios frameworks de persistencia similares a JPA que facilitan la interacción con bases de datos en aplicaciones Java, como son Hibernate, MyBatis, EclipseLink, Spring Data JPA o TopLink.

/ 4. Caso práctico 1: "Entendiendo las bases de datos orientadas a objetos"

Planteamiento: Las bases de datos orientadas a objetos en Java permiten almacenar datos en forma de objetos, similar a como se manejan en la programación orientada a objetos. Estas bases de datos integran directamente con el lenguaje Java, permitiendo que las clases, herencia y otros conceptos se reflejen en la estructura de la base de datos. Las bases de datos relacionales tienen sus filas y sus campos y dentro se guarda la información, pero ¿cómo puede almacenarse la información dentro de las bases de datos orientadas a objetos?

Nudo: ¿Cómo crees que se gestiona la información dentro de una base de datos orientada a objetos? ¿Existirá el concepto de clave primaria o de clave foránea?

Desenlace:

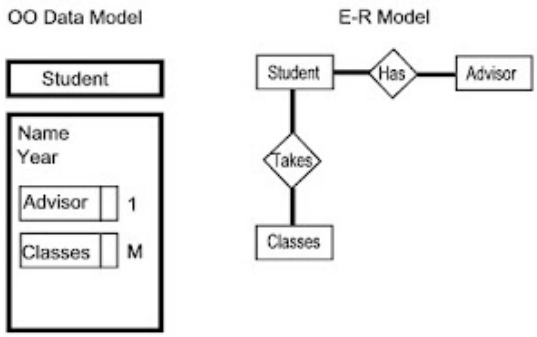


Fig.4. Relación entre entidades.

Es normal que cuando nos topamos con las bases de datos orientadas a objetos no comprendamos muy bien, de primeras, cómo pueden funcionar. Venimos de estudiar una base de datos muy diferente, como es la relacional, que tiene unos conceptos muy claros, como los de integridad de datos, claves primarias y foráneas, relaciones entre tablas, etc.

En las bases de datos orientadas a objetos, lo que se guarda son objetos, y, entre estos, no existe ningún tipo de relación equivalente a las de las bases de datos relacionales, es decir, aquí no hay claves primarias ni foráneas. Los objetos se irán almacenando dentro de la base de datos y su almacenamiento se parecerá más al de un fichero que al de una base de datos.

/ 5. Entidades

Una de las características de **JPA** es que **permite definir clases Java como entidades que representan tablas en la base de datos**. También proporciona formas de definir relaciones entre estas entidades, como relaciones uno a uno, uno a muchos y muchos a muchos. **Para que una clase Java se convierta en una entidad, primero tiene que cumplirse lo siguiente**:

- **Ser un POJO**, es decir, no tener dependencias externas para su funcionamiento.

> ¡! **Sabías que...**
>
> Los POJO son objetos simples y ligeros que no implementan ninguna interfaz ni extienden de ninguna clase específica. No dependen de ninguna tecnología, librería o framework específico y pueden ser fácilmente serializables y deserializables.

- **Tener un constructor vacío** (sin argumentos).
- **Ser una clase de primer nivel**.
- **No estar definida como final**.
- **Implementar Serializable**.

En el caso de que la clase cumpla estos requisitos, podrá ser convertida en entidad. Para ello, se deben cumplir los siguientes requisitos:

- **Anotación @Entity**: La clase debe ser anotada con @Entity para indicar a JPA que la misma es una entidad que se puede almacenar en la base de datos.
- **Identificador único**: La clase debe tener al menos un atributo que actúe como identificador único de la entidad. Este atributo se anota con @Id, indicando que es la clave primaria de la entidad.
- **Generación del valor del identificador**: Opcionalmente, se puede especificar cómo se generará el valor del identificador único. Esto se puede hacer usando anotaciones como @GeneratedValue, que especifica una estrategia de generación de valores automáticamente.
- **Constructor vacío**: La clase debe tener un constructor sin argumentos (constructor vacío). Esto es necesario para que los proveedores JPA puedan instanciar la clase utilizando la reflexión.
- **Mapeo de atributos**: Los atributos de la clase deben ser mapeados a columnas en la tabla de la base de datos. Esto se puede hacer empleando varias anotaciones, como @Column, para especificar el nombre de la columna, su tipo de datos y restricciones adicionales, y otras anotaciones para manejar relaciones entre entidades.
- **Relaciones entre entidades**: Si la entidad tiene relaciones con otras entidades, se deben mapear utilizando anotaciones como @OneToOne, @OneToMany, @ManyToOne o @ManyToMany, según corresponda.

```java
@Entity
public class Alumno implements java.io.Serializable {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nombre;
    private String email;


    public Alumno() {

    }

    //Getter y Setter de los atributos
    // Otros atributos y métodos...
}
```

Fig.5. Entidad Alumno.

/ 6. Unidad de persistencia

Una unidad de persistencia en JPA (*Java Persistence API*) es una **abstracción que representa una fuente de persistencia de datos**, como una base de datos relacional.

En el contexto de JPA, una unidad de persistencia se define mediante un archivo de configuración llamado **persistence.xml** en el que se especifican detalles como el nombre de la unidad de persistencia, las clases de entidad que están siendo gestionadas por esa unidad de persistencia y las propiedades de conexión a la base de datos. **La unidad de persistencia es esencial para que JPA pueda interactuar con la base de datos de manera eficiente y precisa**.

Algunas de las **configuraciones comunes** que se encuentran en el archivo persistence.xml son:

- **Nombre de la unidad de persistencia**: Cada unidad de persistencia se identifica mediante un nombre único. Este se utiliza para hacer referencia a la unidad de persistencia desde el código de la aplicación.
- **Proveedor de persistencia**: Se especifica el proveedor de persistencia JPA que se empleará. Este proveedor puede ser Hibernate, EclipseLink u OpenJPA, entre otros.
- **Clases de entidad**: Se enumeran las clases de Java que actúan como entidades JPA, es decir, las clases que representan tablas en la base de datos. Estas clases serán administradas por la unidad de persistencia y se usarán para realizar operaciones de persistencia.
- **Configuración de la base de datos**: Se especifican las propiedades de conexión a la base de datos, como la URL de conexión, el nombre de usuario y la contraseña. También se pueden configurar otras propiedades relacionadas con el comportamiento de la conexión y el manejo de transacciones.

Un ejemplo simple de un archivo persistence.xml podría ser el mostrado en la figura 6.

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="2.1" xmlns="http://xmlns.jcp.org/xml/ns/persistence" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
                           xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.xsd">
    <persistence-unit name="MiUnidadPersistencia" transaction-type="RESOURCE_LOCAL">
        <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
        <class>modelo.Alumno</class>
        <properties>
            <property name="javax.persistence.jdbc.url" value="jdbc:mysql://localhost:3306/mi_basededatos"/>
            <property name="javax.persistence.jdbc.user" value="usuario"/>
            <property name="javax.persistence.jdbc.password" value="contraseña"/>
            <property name="javax.persistence.jdbc.driver" value="com.mysql.jdbc.Driver"/>
        </properties>
    </persistence-unit>
</persistence>
```

Fig.6. Creación de un archivo persistence.xml.

Este ejemplo configura una unidad de persistencia llamada 'MiUnidadPersistencia' que utiliza el proveedor de persistencia Hibernate y gestiona la clase de entidad Alumno. Las propiedades de conexión a la base de datos están configuradas para una base de datos MySQL local.

Vídeo 1. "Consultando datos con QBE"
https://bit.ly/39llBg3

/ 7. Gestor de entidades

El EntityManager en Java es una interfaz proporcionada por JPA que se utiliza para interactuar con el contexto de persistencia. En otras palabras, el **EntityManager es responsable de gestionar las entidades JPA y las operaciones de persistencia asociadas**, como insertar, actualizar, eliminar y recuperar entidades de la base de datos.

Sin embargo, el EntityManager no se crea directamente con el operador 'new', sino que se obtiene a partir del EntityManagerFactory. En JPA, la relación entre el EntityManager y el EntityManagerFactory es fundamental para entender cómo se gestionan las operaciones de persistencia y cómo se crean y mantienen las instancias de EntityManager.

7.1. EntityManagerFactory

El **EntityManagerFactory es responsable de crear instancias de EntityManager**. Se puede considerar como una fábrica de EntityManagers. Normalmente, solo hay, en toda la aplicación, una instancia de EntityManagerFactory creada al inicio de la aplicación que se mantiene durante toda su vida útil.

El EntityManagerFactory se encarga de la configuración global de la persistencia, como la conexión a la base de datos, el proveedor de persistencia utilizado y otras configuraciones relacionadas con JPA.

7.2. EntityManager

Es **responsable de realizar operaciones de persistencia y manipular entidades dentro del mismo contexto y la base de datos**, para realizar operaciones de persistencia como insertar, actualizar, eliminar y recuperar entidades.

El EntityManager es creado a partir de un EntityManagerFactory utilizando el método createEntityManager().

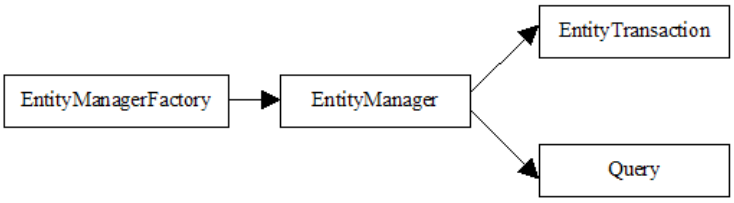


Fig.7. Gestión de entidades.

En resumen, el EntityManagerFactory es responsable de crear instancias de EntityManager, mientras que el EntityManager realiza las operaciones de persistencia y manipula las entidades dentro del contexto de persistencia.

La relación entre ambos es que el EntityManagerFactory crea y mantiene instancias de EntityManager, que a su vez se utilizan para interactuar con la base de datos y realizar operaciones de persistencia en entidades JPA.

7.3. Métodos y transacciones del EntityManager

El EntityManager dispone de varios métodos encargados de gestionar su creación, realizar operaciones de persistencia y consultas, administración de transacciones y eventual cierre para liberar recursos, como muestra la figura 6.

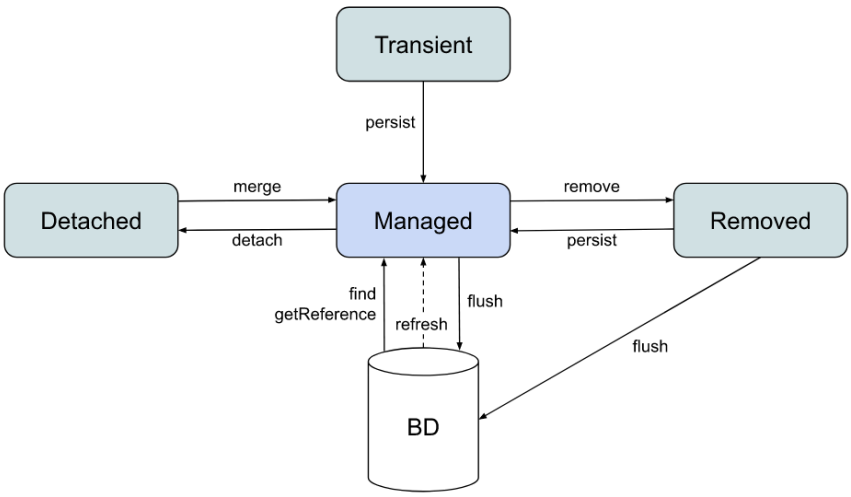


Fig.8. Ciclo de vida de entidades.

Algunos de los **métodos más comunes** son:

- **persist(Objectentity)**: Persiste una nueva entidad en la base de datos.
- **merge(Objectentity)**: Actualiza una entidad existente en la base de datos.
- **remove(Objectentity)**: Elimina una entidad existente de la base de datos.
- **find(Class<T>entityClass, ObjectprimaryKey)**: Recupera una entidad por su clave primaria.
- **createQuery(StringqlString)**: Crea una consulta JPQL (Java Persistence Query Language).
- **getTransaction()**: Obtiene el objeto de transacción asociado al EntityManager.
- **flush()**: Sincroniza el contexto de persistencia con la base de datos.
- **refresh(Objectentity)**: Refresca el estado de una entidad desde la base de datos.
- clear(): Borra el contexto de persistencia, desvinculando todas las entidades gestionadas y borrando la caché de persistencia.
- **close()**: Cierra el EntityManager, liberando los recursos asociados.

Cualquier operación que modifique la base de datos debe realizarse dentro de una transacción y siguiendo estos pasos:

1. Para iniciar una transacción, se debe utilizar el método getTransaction() para obtener el objeto de transacción y luego llamar a begin() en ese objeto.
2. Una vez realizadas todas las operaciones de persistencia necesarias, llamar a commit() en el objeto de transacción para confirmar los cambios en la base de datos.
3. Si algo sale mal o se desea deshacer los cambios realizados en la transacción, llamar a rollback() en el objeto de transacción para revertir los cambios.

/ 8. Operaciones CRUD

Las operaciones CRUD (crear, leer, actualizar y eliminar) son fundamentales en el desarrollo de aplicaciones que manejan datos persistentes. En el contexto de JPA, que es una API estándar de Java para el mapeo objeto-relacional (ORM), estas operaciones se realizan sobre entidades que representan objetos en la base de datos.

8.1. Create

Para **crear un nuevo registro en la base de datos**, primero necesitamos crear una nueva instancia de la entidad correspondiente y luego persistirla en la base de datos, como muestra la figura 9.

```
EntityManagerFactory entityManagerFactory = Persistence.createEntityManagerFactory("MiUnidadPersistencia");
EntityManager entityManager = entityManagerFactory.createEntityManager();
entityManager.getTransaction().begin();

// Crear una nueva instancia de la entidad
Entidad entidad = new Entidad();
entidad.setAtributo1(valor1);
entidad.setAtributo2(valor2);
// Establecer otros atributos

// Persistir la entidad en la base de datos
entityManager.persist(entidad);

entityManager.getTransaction().commit();
entityManager.close();
entityManagerFactory.close();
```

Fig.9. Creación de una entidad.

8.2. Read

Para **leer datos** de la base de datos, se puede utilizar el método find() del EntityManager proporcionado por JPA, como vemos en la figura 10.

```
EntityManagerFactory entityManagerFactory = Persistence.createEntityManagerFactory("MiUnidadPersistencia");
EntityManager entityManager = entityManagerFactory.createEntityManager();

// Leer una entidad por su ID
Entidad entidad = entityManager.find(Entidad.class, id);

// Realizar operaciones con la entidad

entityManager.close();
entityManagerFactory.close();
```

Fig.10. Lectura de una entidad.

8.3. Update

Para **actualizar un registro existente** en la base de datos, primero necesitamos recuperar la entidad, realizar los cambios necesarios y luego persistir esos cambios en la base de datos, como muestra la figura 11.

```
EntityManagerFactory entityManagerFactory = Persistence.createEntityManagerFactory("MiUnidadPersistencia");
EntityManager entityManager = entityManagerFactory.createEntityManager();
entityManager.getTransaction().begin();

// Leer una entidad por su ID
Entidad entidad = entityManager.find(Entidad.class, id);

// Realizar los cambios necesarios en la entidad
entidad.setAtributo1(nuevoValor1);
entidad.setAtributo2(nuevoValor2);
// Establecer otros cambios

// Actualizar la entidad en la base de datos
entityManager.merge(entidad);

entityManager.getTransaction().commit();
entityManager.close();
entityManagerFactory.close();
```

Fig.11. Actualización de una entidad.

8.4. Delete

Para **eliminar un registro** de la base de datos, primero es preciso recuperar la entidad y luego eliminarla, como se puede ver en la figura 12.

```
EntityManagerFactory entityManagerFactory = Persistence.createEntityManagerFactory("MiUnidadPersistencia");
EntityManager entityManager = entityManagerFactory.createEntityManager();
entityManager.getTransaction().begin();

// Leer una entidad por su ID
Entidad entidad = entityManager.find(Entidad.class, id);

// Eliminar la entidad de la base de datos
entityManager.remove(entidad);

entityManager.getTransaction().commit();
entityManager.close();
entityManagerFactory.close();
```

Fig.12. Borrado de una entidad.

En los códigos, 'MiUnidadPersistencia' es el nombre de la unidad de persistencia definida en el archivo persistence.xml.

/ 9. Controlador JPA

Es una **clase** que se utiliza **para manejar la interacción entre la capa de negocio de una aplicación y la capa de persistencia que utiliza JPA para interactuar con la base de datos**. Este controlador actúa como un intermediario que encapsula la lógica relacionada con la persistencia de entidades y proporciona métodos para realizar operaciones CRUD (crear, leer, actualizar, eliminar) en esas entidades. Se trata del nombre con el que se hace referencia al patrón de diseño DAO en JPA.

9.1. Componentes de un controlador JPA

Los principales componentes de un controlador JPA son:

- **EntityManager**: El controlador JPA generalmente tiene una referencia al EntityManager, que es utilizado para realizar operaciones de persistencia en las entidades. El EntityManager es responsable de gestionar el ciclo de vida de las entidades y de ejecutar las consultas JPQL (Java Persistence Query Language).

- **Métodos CRUD**: Un controlador JPA proporciona métodos para realizar operaciones CRUD en las entidades. Estos métodos incluyen guardar una entidad en la base de datos (*persist*), obtener una entidad por su identificador único (*find*), actualizar una entidad existente (*merge*) y eliminar una entidad de la base de datos (*remove*).

- **Unidad de persistencia**: El controlador JPA generalmente está asociado a una unidad de persistencia, que es una agrupación lógica de entidades gestionada por un EntityManagerFactory. La unidad de persistencia está definida en el archivo persistence.xml y contiene la configuración de conexión a la base de datos.

```java
@Repository // Esta anotación indica que esta clase maneja el acceso a la base de datos
public class EntidadJPAController {

    @PersistenceContext
    private EntityManager entityManager; // EntityManager se utiliza para interactuar con la base de datos

    @Transactional // Esta anotación asegura que las operaciones sean realizadas dentro de una transacción
    public void guardarEntidad(Entidad entidad) {
        entityManager.persist(entidad); // Persiste la entidad en la base de datos
    }

    public Entidad obtenerEntidadPorId(Long id) {
        return entityManager.find(Entidad.class, id); // Busca y retorna una entidad por su ID
    }

    @Transactional
    public void actualizarEntidad(Entidad entidad) {
        entityManager.merge(entidad); // Actualiza la entidad en la base de datos
    }

    @Transactional
    public void eliminarEntidad(Long id) {
        Entidad entidad = entityManager.find(Entidad.class, id);
        if (entidad != null) {
            entityManager.remove(entidad); // Elimina la entidad de la base de datos
        }
    }
}
```

Fig.13. Controlador JPA.

En este ejemplo (figura 13):

- La anotación @Repository indica que esta clase maneja el acceso a la base de datos.
- La anotación @PersistenceContext se utiliza para inyectar el EntityManager en esta clase, lo que permite interactuar con la base de datos a través de JPA.
- Los métodos guardarEntidad, obtenerEntidadPorId, actualizarEntidad y eliminarEntidad realizan las operaciones CRUD sobre las entidades.

/ 10. Consulta de entidades: JPQL

JPQL (Java Persistence Query Language) es un **lenguaje de consulta orientado a objetos** utilizado en el **contexto de JPA** para realizar **consultas en bases de datos relacionales**. Es un lenguaje similar a SQL, pero que, en lugar de operar sobre tablas y columnas de la base de datos, lo hace sobre entidades y atributos de objetos Java.

Las **características principales** de JPQL son:

- **Orientado a objetos**: Opera sobre entidades y atributos de objetos Java en lugar de tablas y columnas de la base de datos. Esto permite realizar consultas utilizando la estructura de clases y relaciones definidas en el modelo de objetos de la aplicación.
- **Independiente del proveedor**: Es independiente del proveedor de JPA, lo que significa que las consultas escritas en JPQL son portables entre diferentes proveedores de bases de datos. Esto permite escribir consultas una vez y ejecutarlas en cualquier base de datos compatible con JPA.
- **Consulta basada en entidades**: Permite realizar consultas basadas en entidades y relaciones entre entidades. Por ejemplo, se pueden efectuar consultas para recuperar todas las instancias de una entidad, filtrar instancias basadas en ciertos criterios, realizar uniones entre entidades relacionadas y hacer agregaciones de datos.

- **Sintaxis similar a SQL**: Aunque JPQL es orientado a objetos, su sintaxis está inspirada en SQL, lo que facilita a los desarrolladores familiarizados con SQL escribir consultas en JPQL. Sin embargo, JPQL proporciona características adicionales que no están disponibles en SQL estándar, como la navegación de objetos y las expresiones de objeto.

```
String jpql = "SELECT e FROM Empleado e WHERE e.departamento.nombre = :nombreDepartamento";
Query query = entityManager.createQuery(jpql);
query.setParameter("nombreDepartamento", "Ventas");
List<Empleado> empleados = query.getResultList();
```

Fig.14. Ejemplo de consulta JPQL.

En este ejemplo (figura 14), la consulta JPQL selecciona todos los empleados cuyo departamento tiene el nombre 'Ventas'.

Veamos los **beneficios** de JPQL:

- **Abstracción de la base de datos**: Permite a los desarrolladores escribir consultas utilizando la estructura de clases y relaciones definidas en el modelo de objetos de la aplicación, lo que proporciona una capa de abstracción sobre la base de datos subyacente.

- **Portabilidad**: Las consultas escritas en JPQL son portables entre diferentes proveedores de bases de datos compatibles con JPA, lo que facilita el cambio de base de datos sin tener que modificar las consultas.

- **Legibilidad**: Ofrece una sintaxis clara y legible que facilita la escritura y comprensión de consultas, especialmente para desarrolladores familiarizados con SQL.

/ 11. Caso práctico 2: "¿Se puede seguir empleando SQL?"

Planteamiento: Las bases de datos orientadas a objetos, como DB4O y ObjectDB, tienen sus propias API de consulta, que están diseñadas para trabajar directamente con objetos Java, proporcionando una manera más natural y eficiente de interactuar con los datos en un entorno orientado a objetos.

Nudo: ¿Crees que se podrá utilizar el lenguaje SQL para poder operar con bases de datos orientadas a objetos?

Desenlace: No es común emplear SQL en bases de datos orientadas a objetos (OODB) en Java, ya que estas emplean su propio lenguaje de consultas orientado a objetos, más adecuado para manejar objetos complejos y sus relaciones.

Sin embargo, existen muchos gestores de bases de datos que soportan las bases de datos orientadas a objetos y permiten algún nivel de integración con SQL, aunque esto suele ser limitado y no aprovecha todas las características orientadas a objetos.



Fig.15. Gestores de bases de datos.

Un caso sería DB4O, pero existen gestores de bases de datos como Oracle y Microsoft SQL Server que, además de trabajar con bases de datos relacionales, soportan las bases de datos orientadas a objetos. En estos casos, sí existe una extensión de SQL, por llamarla así, que permite trabajar con objetos, aunque no es exactamente igual que el SQL que trabaja con bases de datos relacionales.

/ 12. Consultas JPQL

Son similares a las consultas SQL, pero están diseñadas **específicamente para trabajar con entidades persistentes en el contexto de JPA**.

> **¡! Sabías que...**
>
> Con JPQL, los desarrolladores pueden escribir consultas que son independientes del proveedor de JPA subyacente. Esto significa que una consulta JPQL puede funcionar con cualquier implementación de JPA (como Hibernate, EclipseLink, etc.).

12.1. Select

Una sentencia SELECT es una instrucción utilizada en lenguajes de consulta de bases de datos para **recuperar datos de una o más tablas**. Básicamente, permite realizar consultas a una base de datos para obtener información específica según ciertos criterios.

A continuación, se va a realizar una descripción de cómo funciona una consulta SELECT en JPQL:

1. **Definir la consulta**: Primero, se define la consulta JPQL como una cadena de texto en el código Java. Por ejemplo:

```
Stringjpql = "SELECT e FROM Empleado e WHERE e.departamento.nombre = :nombreDepartamento";
```

2. **Crear el EntityManager y la transacción**: Antes de ejecutar la consulta, se necesita obtener un EntityManager y comenzar una transacción si se está trabajando en un contexto transaccional.

```
EntityManagerentityManager = entityManagerFactory.createEntityManager();
EntityTransaction transaction = entityManager.getTransaction();
transaction.begin();
```

3. **Crear y ejecutar la consulta**: Se utiliza el EntityManager para crear una consulta JPQL utilizando el método createQuery() y luego se ejecuta utilizando el método getResultList() o getSingleResult(), según corresponda.

```
TypedQuery<Empleado> query = entityManager.createQuery(jpql, Empleado.class);
query.setParameter("nombreDepartamento", nombre);
List<Empleado>employees = query.getResultList();
```

4. **Gestionar los resultados**: Es posible obtener los resultados de la consulta en una lista [en el caso de getResultList()] o como un solo resultado [en el caso de getSingleResult()].

5. **Cerrar la transacción y el EntityManager**: No hay que olvidarse de hacer un *commit* de la transacción y cerrar el EntityManager cuando se haya terminado.

```
transaction.commit();
entityManager.close();
```

```
String jpql = "SELECT e FROM Empleado e WHERE e.departamento.nombre = :nombreDepartamento";

EntityManager entityManager = entityManagerFactory.createEntityManager();
EntityTransaction transaction = entityManager.getTransaction();
transaction.begin();


TypedQuery<Empleado> query = entityManager.createQuery(jpql, Empleado.class);
query.setParameter("nombreDepartamento", nombre);
List<Empleado> employees = query.getResultList();

transaction.commit();
entityManager.close();
```

Fig.16. Ejemplo de consulta Select.

12.2. Consultas Update y Delete

Por supuesto, en JPQL también es posible realizar operaciones de actualización y eliminación de datos, similar a como se hace en SQL.

Las consultas se efectuarán empleando la misma metodología y lo único que cambiará será la sentencia jpql:

```
Stringjpql = "UPDATE Empleado e SET e.salario = :nuevoSalario WHERE e.departamento = :dept";
```

En este código se actualiza el salario de los empleados que pertenecen a un departamento específico.

```
Stringjpql = "DELETE FROM Empleado e WHERE e.departamento = :dept";
```

Esta consulta elimina todos los empleados que pertenezcan al departamento especificado.

12.3. Otras consultas

Consulta básica SELECT: Selecciona todos los objetos de una entidad específica.

```
String jpql = "SELECT e FROM Employee e";
```

Consulta con condiciones WHERE: Selecciona objetos que cumplan ciertas condiciones.

```
String jpql = "SELECT e FROM Employee e WHERE e.department = :dept";
```

Consulta con ordenamiento ORDER BY: Ordena los resultados según una o más propiedades.

```
String jpql = "SELECT e FROM Employee e ORDER BY e.salary DESC";
```

Consulta con parámetros: Utiliza parámetros para hacer consultas dinámicas.

```
String jpql = "SELECT e FROM Employee e WHERE e.department = :dept AND e.salary> :minSalary";
```

Consulta con funciones de agregación: Realiza operaciones de agregación como SUM, COUNT, AVG, etc.

```
String jpql = "SELECT AVG(e.salary) FROM Employee e WHERE e.department = :dept";
```

Consulta con JOIN: Une entidades relacionadas para obtener información más completa.

```
String jpql = "SELECT e FROM Employee e JOIN e.department d WHERE d.name = :deptName";
```

Consulta con JOIN FETCH: Realiza un JOIN para recuperar datos de la entidad relacionada en la misma consulta.

```
String jpql = "SELECT e FROM Employee e JOIN FETCH e.department WHERE e.department = :dept";
```

Consulta con funciones de fecha: Utiliza funciones de fecha para realizar consultas basadas en fechas.

```
String jpql = "SELECT e FROM Employee e WHERE e.startDate> :startDate";
```

/ 13. Resumen y resolución del caso práctico de la unidad

A lo largo de este tema, se continúa con el estudio de las bases de datos orientadas a objetos que hacen uso de frameworks de persistencia encargados de manejar los objetos.

Se ha visto cómo realizar las operaciones típicas de toda base de datos con este nuevo framework y cómo insertar, actualizar, borrar y consultar datos con diferentes criterios de búsqueda añadiendo esta nueva capa de abstracción. Asimismo, se han tratado las sentencias nativas de SQL adaptadas al uso del framework.

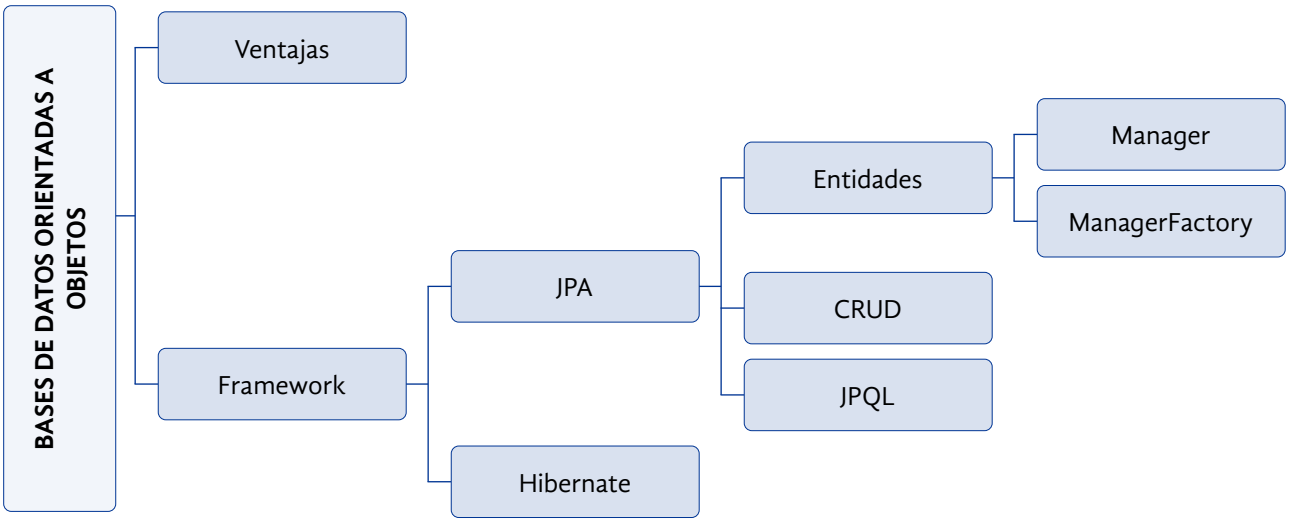


Fig.17. Esquema resumen del tema.

Resolución del caso práctico de la unidad

Con respecto a las bases de datos, ya sabemos que existen multitud de tipos diferentes a las relacionales. Un ejemplo son las que hemos estudiado en el tema: las orientadas a objetos.

Cuando conocemos varios tipos de bases de datos, nos surge la pregunta de cuándo tenemos que utilizar uno u otro.

El uso de un tipo de base de datos u otro va a depender casi exclusivamente de la aplicación a desarrollar. Por ejemplo, en el caso de un programa que gestione la información que se produce de un centro educativo, lo más normal sería usar una base de datos relacional, ya que nos facilitará, en gran medida, las operaciones que tengamos que realizar. No obstante, habría que tener cuidado, ya que si tenemos un volumen muy elevado de datos a gestionar, las bases de datos relacionales pueden volverse lentas.

En el caso de las bases de datos orientadas a objetos, se usan principalmente cuando no hay una gran cantidad de información a gestionar y, además, necesitamos una base de datos portable independiente de la plataforma.

/ 14. Bibliografía

Bhargava, A. Y. (2019). *Algoritmos: Una guía ilustrada para programadores y curiosos*. Madrid: Ediciones Anaya Multimedia (Grupo Anaya).

Bloch, J. (2018). *Effective Java* (3.ª ed.). Addison-Wesley.

Caro, C. M., Ramos, A. N. y Barceló, A. V. (2002). *Introducción a la programación con orientación a objetos*. Prentice Hall.

Knuth, D. E. (1997). *The Art of Computer Programming* (Vol. 1: *Fundamental Algorithms*) (3.ª ed.). Addison-Wesley.

Ojeda, F. C. (2001). *Introducción a la programación*. Anaya Multimedia.

Schildt, H. (2018). *Java: The complete reference* (11.ª ed.). McGraw-Hill Education.